Ariel University

Advanced Quantization Methods for Accelerating Machine Learning

Authors: Noam Cohen, Bar Yechezkel and Yair Horovitz
This project is submitted as part of the requirements for the degree of a B.Sc. in Computer Science

Supervisor: Itamar Cohen, Ph.D.

October 19, 2025

Abstract

Deep neural networks achieve impressive results across a variety of tasks, but their high computational and memory requirements make them difficult to deploy efficiently, especially on edge devices with limited resources. Quantization – the process of representing numbers with fewer bits – is a widely used technique to reduce resource consumption while maintaining acceptable *accuracy.*

In this work, we systematically evaluate different quantization schemes and analyze their impact on model performance. Specifically, we examine combinations of symmetry type (*symmetric* versus *asymmetric* quantization), bit-width (8-bit and 16-bit), and clipping methods, where clipping can be based on fixed percentiles, standard deviation thresholds, or not applied at all. By considering all possible configurations of these factors, we aim to identify the setup that best balances efficiency and accuracy, minimizing memory and compute costs while preserving predictive performance.

Contents

1 Introduction **3**

2 Literature Review **5**

2.1 Quantization Techniques for Neural Networks 5

2.1.1 Quantization Fundamentals 5

2.1.2 Quantization Approaches: QAT vs PTQ 6

2.1.3 Quantization Schemes 6

2.1.4 Practical Applications 7

2.1.5 Advanced Quantization Techniques and Complementary Efficiency Methods 7

2.1.6 Practical Considerations 8

3 Mathematical Model and Notation **9**

3.1 Floating-Point Representation 9

3.2 Quantization Notation 9

3.3 Quantization and Dequantization 10

3.4 Quantized Accumulation in Hardware 11

3.5 Quantized Layer Inference 12

3.6 Clamping Strategies in Quantization 13

3.7 Algorithm 14

4 Simulation and Evaluation Settings **15**

4.1 Dataset 15

4.2 Quantization Configurations 15

4.3 Evaluation Metrics 16

4.4 Experimental Protocol 17

4.5 Implementation Details 17

5 Results and Evaluation **19**

5.1 Quantization Results 19

5.2 MAE Results 19

5.3 MSE Results 19

5.4 Accuracy Results 20

5.5 Visualization 20

5.6 Discussion of Results 21

6 Conclusions and Future Work **23**

1 Introduction

Modern neural networks have achieved leading results across a wide range of tasks, but this success comes with extremely high computational and memory demands. Running such models requires considerable hardware resources and time, which becomes especially problematic when deploying them on edge devices such as smartphones, IoT hardware, or embedded systems with limited capacity. As a result, improving the efficiency of neural networks has become a crucial research direction.

A key reason for these high computational demands is that neural networks are fundamentally built upon *multiply-accumulate(MAC)* operations. Each neuron computes a weighted sum of its inputs, which requires a large number of multiplications followed by accumulations. Deep networks can contain millions to billions of such operations, and the cost of performing MACs grows rapidly with model size. Since MAC units dominate both the arithmetic logic and the energy consumption in modern accelerators [3], any method that reduces their precision or simplifies their implementation can yield significant improvements in efficiency.

One of the most effective techniques for reducing resource consumption is quantization - the process of representing numbers with fewer bits. Weights and activations are a common example of this principle. By lowering numerical precision (e.g., from 32-bit floating point to 16-bit or 8-bit integers), models can achieve faster execution and reduced memory usage. However, quantization inevitably introduces approximation errors, as many high-precision values are mapped into fewer discrete levels, potentially reducing model accuracy. The challenge, therefore, is to design quantization schemes that achieve the best balance between efficiency and accuracy.

To illustrate this idea in an intuitive way, consider representing real numbers between 0 and 1000, but instead of keeping every possible value, we allow only a very coarse set of representative points (as if we had only a single digit to store them). The range is divided into broad intervals, and each interval is mapped to a representative value, as shown in Table 1. However, if most values lie in a narrow region (e.g., [500,510]) while only a few extreme outliers exist (e.g., 0 and 999), one may decide to *clamp* these outliers to save precision for the dense region.

An Intuitive Example: Outliers and Clamping

Consider the vector $x = \{1, 2, 2, 3, 3, 3, 4, 4, 100\}$. Most values lie in the range 1–4, but a single outlier (100) stretches the dynamic range. If we quantize the full interval $[1, 100]$, many integer levels are "wasted" on the tail, leaving poor resolution where most of the data live. A simple remedy is to *clamp* extreme values before quantization so that the available levels are concentrated on the typical region. The formal ways to choose clamping bounds (percentiles or $\pm k\sigma$) appear later in Sec. 3.6.

Quantization methods can differ along several dimensions. First, they may vary in the number of bits used to represent numbers. Second, they may differ in how they handle outliers – values that fall far outside the typical distribution range. Finally, they can

Table 1: Intuitive quantization example: coarse intervals vs. clamping of outliers. In Case 1 the intervals are spread uniformly across the full range [0,1000], while in Case 2 we clamp the outliers (0,999) and allocate more quantization levels to the dense region [500–510].

Case 1: Uniform quantization over [0,1000]	
Input Range	**Quantized Value**
$[0, 50]$	0
$[50, 150]$	100
$[150, 250]$	200
$\vdots$	$\vdots$
$[950, 1000]$	950
Case 2: Data mostly in [500,510], with outliers {0,999}	
Input Range	**Quantized Value**
$[500, 502]$	500
$[502, 504]$	502
$[504, 506]$	504
$[506, 508]$	506
$[508, 510]$	508
Outliers $\{0, 999\}$	Clamped to nearest range

be distinguished by how they implement arithmetic operations under reduced precision. These criteria form the basis for comparing quantization strategies in this work.

Scope of This Work

Building upon prior research, this project, titled **Advanced Quantization Techniques for Accelerating Machine Learning**, is dedicated to a systematic evaluation of practical quantization methods applied to convolutional neural networks. In particular, we focus on integer-based approaches (*INT8* and *INT16*) that are highly relevant for real-world deployment on resource-constrained devices. We investigate symmetric vs. asymmetric schemes, combined with multiple clamping strategies (percentile and standard deviation based), in order to evaluate their trade-offs in terms of memory, computation, and accuracy. Our experiments are conducted on a pretrained *ResNet-18* evaluated over a balanced subset of the Animals10 dataset [4], ensuring reproducible comparisons across all quantization configurations.

2 Literature Review

2.1 Quantization Techniques for Neural Networks

Efficient deployment of neural networks on resource-constrained devices, such as mobile phones, embedded systems, and IoT devices, requires careful management of computation, memory, and energy usage. Neural networks are typically implemented using floating-point arithmetic, most commonly in the single-precision format (FP32) defined by the IEEE 754 standard [7]. Floating-point formats allow a wide dynamic range and good numerical precision, which makes them the default choice in scientific computing, computer graphics, and machine learning.

At the same time, floating-point arithmetic is relatively complex and power-hungry. The partition into mantissa and exponent, while enabling flexibility and range, also makes the underlying hardware circuits for addition and multiplication significantly more involved compared to fixed-point or integer arithmetic. As networks grow in size and complexity, these costs become a major obstacle for real-time deployment on low-power devices. Quantization has therefore emerged as one of the most effective strategies for reducing these costs while maintaining acceptable accuracy levels [1, 2, 3]. In line with this trend, NVIDIA introduced the TensorFloat-32 (TF32) format in its A100 GPUs, achieving up to 20× speedups in AI training and high-performance computing workloads by lowering precision while preserving accuracy [8].

2.1.1 Quantization Fundamentals

Quantization is a key technique for improving the efficiency of neural networks, particularly when deploying them on resource-constrained devices. It reduces the precision of weights and activations, typically mapping 32-bit floating-point values (FP32) to lower-bit fixed-point or integer representations, such as 8-bit integers (INT8). This conversion significantly decreases memory requirements- by a factor of four when moving from FP32 to INT8- and reduces computational cost for matrix multiplications quadratically, by a factor of sixteen [3].

Low-bit quantization introduces rounding noise, as FP32 values (already discrete, though with very fine resolution) are mapped to a coarser discrete set of INT8 levels. A grid in this context refers to that finite set of discrete points, representing the only valid numbers available after quantization. Instead of a nearly continuous FP32 range, values are projected onto this limited integer grid.

While the Introduction provided an intuitive illustration of quantization, the following demonstrates a concrete numerical case of FP32 quantization to INT8, highlighting how real-valued inputs are approximated within a limited integer grid.

2.1.2 Quantization Approaches: QAT vs PTQ

Before discussing specific quantization strategies, it is useful to briefly recall how neural networks operate. The **training phase** is the process in which the network parameters (weights and biases) are optimized on large datasets using gradient-based methods. Training is computationally expensive, often requiring specialized hardware such as GPUs or TPUs, and is considered a major contributor to *capital expenditure (CapEx)*, since it largely determines the time-to-market of AI models [9]. The **inference phase**, in contrast, is the deployment of a trained network to provide predictions for new inputs. Inference must be efficient to ensure fast response times, especially in interactive applications, and therefore contributes directly to *operational expenditure (OpEx)*, i.e., the recurring cost per query during real-world usage [9].

Two main regimes for quantization have been proposed in the literature: *Quantization-Aware Training (QAT)* and *Post-Training Quantization (PTQ)* [3].

Quantization-Aware Training (QAT): QAT integrates quantization into the training process by simulating low-precision arithmetic during weight updates. This allows the network to adapt its parameters to quantization noise, resulting in higher accuracy for aggressive quantization (e.g., fewer than 8 bits) [3]. The downside is the additional computational cost and training complexity, as QAT requires retraining or fine-tuning the network. It is particularly useful for applications where maintaining accuracy is critical, such as computer vision tasks in medical imaging or autonomous driving [3].

Post-Training Quantization (PTQ): PTQ applies quantization to a pre-trained network without retraining. It is simple to implement and requires minimal engineering effort, making it a popular choice for deploying existing models on edge devices. PTQ works well for 8-bit quantization in many cases, often achieving negligible accuracy loss [3]. However, for very low-bit quantization (e.g., 4-bit or mixed-precision), PTQ can cause significant accuracy degradation, especially for networks with sensitive layers or highly non-uniform weight distributions [3].

Practical Considerations: In practice, PTQ is often the first choice for quick deployment with minimal accuracy loss. When lower-bit quantization or highly optimized performance is required, QAT is preferred. Some pipelines even use a hybrid approach, applying PTQ to most layers while using QAT selectively for sensitive layers [3].

2.1.3 Quantization Schemes

Several quantization schemes have been explored to balance accuracy and efficiency:

(The mathematical formulation of these schemes and their computational flow appears later in Section 3.)

Orthogonal decisions. Before choosing a concrete scheme, it is useful to lay out the main *orthogonal* design axes:

1. **Training regime:** *QAT* vs. *PTQ*.
2. **Mapping:** *symmetric* vs. *asymmetric (affine)* quantization.
3. **Clamping strategy:** none vs. percentile- or $k\sigma$-based bounds (where σ is the standard deviation).
4. **Bit-width:** e.g., INT8 vs. INT16.

We next detail the most common mappings and their implications.

- **Symmetric Quantization:** The *zero-point* is fixed at zero, simplifying arithmetic operations. However, it can underutilize the dynamic range if the original distribution is not centered around zero.
- **Asymmetric Quantization (Uniform Affine):** Introduces a zero-point offset to better map the floating-point distribution to the integer grid, reducing quantization error in non-centered distributions [2].
- **Power-of-Two Quantization:** A special case of symmetric quantization where the *scale factor* is restricted to powers of two, allowing hardware-efficient scaling via bit-shifting [3].

  Why is this efficient? If the scale s is constrained to a power of two ($s = 2^k$), then multiplying or dividing by s can be implemented as an integer bit shift (left/right) instead of a general multiply/divide. For example, $37 \cdot 2^3 = 296$ (i.e., `37 << 3`) and $296/2^4 = 18.5$, while in integer arithmetic the operation `296 >> 4` yields 18 due to truncation. Bit shifting is cheaper in hardware (area, power, and latency), hence the efficiency.

2.1.4 Practical Applications

In practice, quantization forms an essential part of model efficiency pipelines for real-world applications. Nearly all deep learning models deployed on mobile devices, embedded systems, or IoT devices use some form of quantization, often reducing precision to INT8 while maintaining close-to-floating-point performance.

2.1.5 Advanced Quantization Techniques and Complementary Efficiency Methods

Recent research has explored a variety of advanced techniques to improve the efficiency of neural networks while minimizing accuracy loss. While not a quantization method per se, *Floating-Floating-Point (F2P)* [2] introduces a novel number representation system. F2P dynamically partitions the mantissa and exponent bits, allowing flexible trade-offs between dynamic range and precision. This is particularly beneficial for networks with skewed weight distributions or short-tail activations, as it enables precise representation of both small and large values. Although distinct from quantization, formats like F2P

can be integrated into quantization pipelines to complement existing approaches. For instance, F2P unifies the benefits of FP8 configurations (e.g., 5M2E and 2M5E) while mitigating their respective limitations [2].

In large language models (LLMs), *QA-LoRA [5] (Quantization-Aware Low-Rank Adaptation)* combines low-rank adaptation with quantization-aware training. QA-LoRA fine-tunes directly on quantized weights (e.g., INT4) while introducing *group-wise operators* that reduce quantization error by assigning scaling and zero-point parameters per group. This maintains parameter efficiency, lowers memory and compute costs, and yields a quantized model ready for deployment without additional post-training steps [5].

Other notable methods include **mixed-precision quantization**, **advanced clipping strategies**, and **per-group quantization** [3], which allow fine-grained control over trade-offs between accuracy, hardware efficiency, and memory usage. Together, these demonstrate the potential of combining flexible numerical representations with efficient adaptation strategies to deploy high-performance models on resource-constrained devices.

Beyond quantization, architectural strategies also contribute to efficiency. [6] introduced the *Mixture-of-Experts (MoE)* paradigm, in which multiple specialized subnetworks ("experts") exist, but only a subset is activated for each input. This design enables massive parameter counts with relatively low computation.

Building on this concept, the same work proposed the *Sparse Mixture-of-Experts (SMoE)* variant, which activates only a tiny fraction of experts, thereby improving efficiency in both memory and computation.

Finally, **SMoE-Dropout [6]** applies a fixed, randomly initialized router that selects experts in a structured-dropout fashion, mitigating representational collapse. This results in a "self-slimmable" transformer whose active experts can be dynamically adjusted during inference or fine-tuning, providing a flexible trade-off between accuracy and efficiency.

2.1.6 Practical Considerations

Implementing quantized networks efficiently requires careful attention to hardware constraints. Fixed-point accelerators, commonly used for INT8 inference, rely on Multiply-Accumulate (MAC) operations, where data transfer and computation dominate energy consumption. By using lower-bit representations, energy usage and latency can be significantly reduced, but the choice of quantization parameters (scale, zero-point, bit-width) directly impacts network performance [3]. Moreover, quantized networks may require calibration, rounding strategies, and selective layer-wise adjustments to preserve accuracy [2, 3].

3 Mathematical Model and Notation

In this section, we introduce the mathematical notation used throughout this report and formalize the computation flow of quantized neural network layers, focusing on both **symmetric** and **asymmetric (affine)** quantization.

3.1 Floating-Point Representation

In general, a floating-point number can be expressed as:

$$N = (-1)^s \cdot (1.m) \cdot 2^{E-Bias},$$

where s is the sign bit, m is the mantissa (fraction) represented with M bits, and E is the exponent represented with E bits.

As a concrete example, the IEEE 754 single-precision (FP32) format partitions the 32-bit word as follows:

- **Sign bit (1 bit):** specifies the polarity of the number (positive or negative).
- **Exponent (8 bits):** encodes the scale of the number, enabling representation across a wide dynamic range.
- **Mantissa/Fraction (23 bits):** stores the significant digits, yielding approximately 7–8 decimal digits of precision.

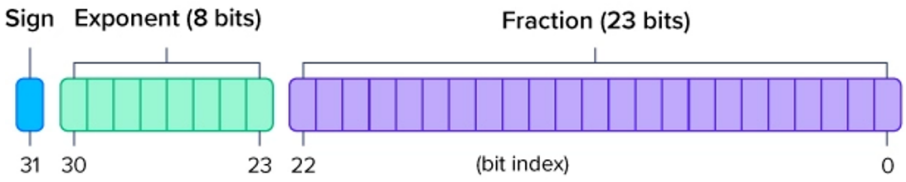


Figure 1: 32-bit floating point (FP32) representation structure.

3.2 Quantization Notation

Table 2 summarizes the symbols used throughout the paper to describe quantization.

Throughout this report, x denotes the real-valued activations entering a layer, and W the layer's weight matrix. Bias terms are represented by b. The quantization function $Q(\cdot)$ maps these real values into discrete integers t_{int} using a chosen bit-width B and scale factors s. The symbol q is used to denote a generic quantized value produced by this mapping. When dequantized, these integers yield $\hat{t}$, an approximation of the original value. Scale factors (s_x, s_W) control the resolution of activations and weights, while zero-points (z_x, z_W) ensure that ranges not centered at zero are represented efficiently. Intermediate integer accumulations (y_{int}) are later reconstructed into floating-point outputs $\hat{y}$ for further processing.

Table 2: Summary of notation used throughout the paper.

Symbol	Meaning
x	Real-valued input (activation)
W	Weight matrix
b	Bias vector
q	Quantized value
$Q(\cdot)$	Quantization function
s, s_x, s_W	Scale factors (general / activations / weights)
z, z_x, z_W	Zero-points (general / activations / weights)
B	Bit-width (e.g., 8, 16)
t_{int}	Quantized integer tensor
$\hat{t}$	Dequantized (reconstructed) value
y_{int}	Integer accumulation result
$\hat{y}$	Final dequantized output

3.3 Quantization and Dequantization

Let $x \in \mathbb{R}^n$ denote an input activation vector, and $W \in \mathbb{R}^{m \times n}$ the weight matrix of a given layer. In full precision, the output is:

$$y = Wx + b, \tag{1}$$

where b is the bias vector.

Quantization maps floating-point values into a discrete integer grid. Given a bit-width b, we define the grid as either signed $(-2^{b-1}, \ldots, 2^{b-1} - 1)$ or unsigned $(0, \ldots, 2^b - 1)$.

Symmetric Quantization Symmetric quantization represents the simplest way to map floating-point values to integers. In this technique, the quantized range is symmetric around zero, typically $[-2^{b-1}, 2^{b-1} - 1]$.

$$t_{\text{int}} = \text{clamp}\left(\text{round}\,\frac{t}{s},\ -2^{b-1},\ 2^{b-1} - 1\right), \tag{2}$$

$$\hat{t} = s \cdot t_{\text{int}}, \tag{3}$$

where:

- $s = \frac{\max(|t_{\min}|, |t_{\max}|)}{2^{b-1}-1}$ is the **scale factor**, which determines the step size of each integer level.
- Zero-point $z = 0$, meaning all values are centered symmetrically around zero.

Illustration:

```
Floating-point:   -5        0         5
Integer grid:     -127      0         127   (b=8)
```

Use case: This method is suitable when data is centered around zero and contains both positive and negative values. It is simple and computationally efficient.

Asymmetric (Affine) Quantization Asymmetric quantization allows mapping data that is not centered around zero. It introduces a **zero-point** z that shifts the integer range such that the minimum value $t_{\min}$ maps to 0 and the maximum value $t_{\max}$ maps to $2^b - 1$.

$$t_{\text{int}} = \text{clamp}\Big(\text{round}\,\frac{t}{s} + z,\ 0,\ 2^b - 1\Big), \tag{4}$$

$$s = \frac{t_{\max} - t_{\min}}{2^b - 1}, \quad z = \text{round}\,-\frac{t_{\min}}{s}, \tag{5}$$

$$\hat{t} = s \cdot (t_{\text{int}} - z). \tag{6}$$

Illustration:

```
Floating-point:    1         5         9
Integer grid:      0         127       255   (b=8)
Zero-point z\approx 28
```

Use case: Asymmetric quantization is useful when data is not centered around zero, e.g., positive-only ranges. It maximizes bit-width utilization and reduces quantization error for non-symmetric ranges.

Key Differences:

Feature	Symmetric	Asymmetric (Affine)
Zero-point z	0	Computed from $t_{\min}$
Integer range	$[-2^{b-1}, 2^{b-1} - 1]$	$[0, 2^b - 1]$
Best for	Data around zero	Non-symmetric ranges
Computational simplicity	High	Slightly more complex

3.4 Quantized Accumulation in Hardware

Figure 2 illustrates a schematic overview of matrix-multiply logic in neural network accelerator hardware. To move from floating-point to efficient fixed-point operations, a floating-point input vector x is approximated by scaling an integer vector x_{int}:

$$\hat{x} = s_x \cdot x_{\text{int}} \approx x, \tag{7}$$

where s_x is the scale factor.

By applying quantization to both weights and activations, the accumulation in layer n can be expressed as:

$$\begin{aligned} \hat{A}_n &= \hat{b}_n + \sum_m \hat{W}_{n,m}\hat{x}_m \\ &= \hat{b}_n + \sum_m \left(s_w W^{\text{int}}_{n,m}\right)\left(s_x x^{\text{int}}_m\right) \\ &= \hat{b}_n + s_w s_x \sum_m W^{\text{int}}_{n,m} x^{\text{int}}_m \end{aligned} \tag{8}$$

where s_w and s_x are the scale factors for weights and activations, respectively, and $W^{\text{int}}, x^{\text{int}}$ are their integer representations. This formulation highlights how efficient integer multiply-accumulate (MAC) units can be used in accelerators to perform quantized inference [3].

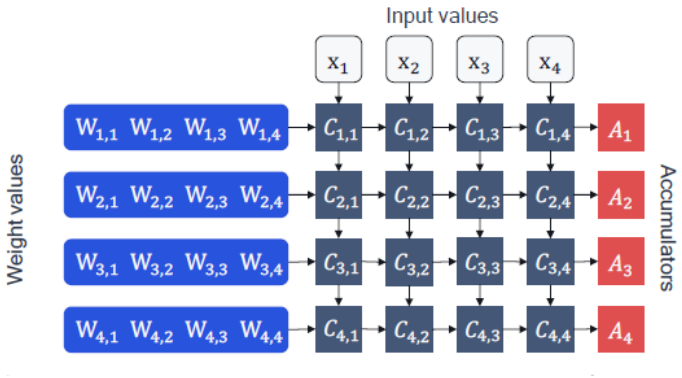


Figure 2: Schematic overview of matrix-multiply logic in neural network accelerator hardware, taken from [3].

3.5 Quantized Layer Inference

During inference, the goal is to compute the layer output efficiently using quantized values. Let x_{int} and W_{int} denote the integer representations of the input activations and weights, obtained after the quantization process described earlier. The layer performs a standard dot product (or convolution) in the integer domain:

$$y_{\text{int}} = \sum_{i=1}^{n} W_{\text{int},i} \cdot x_{\text{int},i}, \tag{9}$$

where y_{int} is the accumulated integer result. This integer computation can be efficiently implemented on hardware accelerators using integer multiply-accumulate (MAC) units, which consume less energy and require less memory bandwidth than floating-point operations.

After computing the integer accumulation, the final floating-point output is reconstructed through dequantization:

$$\hat{y} = s_W \cdot s_x \cdot y_{\text{int}} + b, \tag{10}$$

where s_W and s_x are the scale factors for the weights and input activations, respectively, and b is the bias vector (kept in full precision). This step maps the integer result back to the original floating-point range, allowing the quantized layer output $\hat{y}$ to be used in subsequent layers or for evaluation purposes.

3.6 Clamping Strategies in Quantization

Quantization reduces the precision of weights and activations to improve efficiency, especially on edge devices. However, extreme values (outliers) can dominate the dynamic range, leading to poor resolution for the majority of values. To mitigate this, we apply a **clamping strategy** before quantization.

Formal Definition. For a vector $x \in \mathbb{R}^n$, the clamped version $\tilde{x}$ is defined as:

$$\tilde{x}_i = \min(\max(x_i, L), U)\,, \tag{11}$$

where L and U are the lower and upper bounds of the clamping range, respectively.

Clamping Options. Two common strategies are used:

- **Percentile-based Clamping:** L and U correspond to chosen percentiles of x, e.g., 1% and 99%. This reduces the impact of outliers and focuses the quantization levels on the typical values.
- **Standard Deviation Clamping:** $L = \mu - k\sigma$ and $U = \mu + k\sigma$, where μ is the mean of x, σ is the standard deviation, and k is a tunable multiplier (e.g., $k = 3$). This preserves most of the distribution while limiting extreme values.

Illustrative Example Consider the vector

$$x = \{1, 2, 2, 3, 3, 3, 4, 4, 100\}.$$

The raw dynamic range is clearly dominated by the single outlier 100, yielding

$$[\min(x), \max(x)] = [1, 100].$$

Percentile Clamping. Using the 1% and 99% percentiles, we obtain

$$P_1 \approx 1.08, \quad P_{99} \approx 92.32,$$

which results in an effective range of

$$[1.08,\ 92.32].$$

3σ Clamping. The mean and standard deviation of x are

$$\mu \approx 13.56, \quad \sigma \approx 30.58.$$

Applying a 3σ rule yields

$$[\mu - 3\sigma,\ \mu + 3\sigma] \approx [-78.17,\ 105.28].$$

Since negative values are not relevant in this context, the practical clamped range becomes

$$[1,\ 100].$$

This procedure suppresses rare extreme values, ensuring that the quantization grid is allocated to represent the majority of the data with higher fidelity.

These clamping strategies are applied across both symmetric and asymmetric quantization and for 8-bit and 16-bit representations, as summarized in Table 3.

3.7 Algorithm

Algorithm 1 Quantized Inference Algorithm

Require: Input x, weights $W^{(l)}$, biases $b^{(l)}$ for $l = 1, \ldots, L$
Require: Bit-width b, per-layer scales $s_x^{(l)}, s_W^{(l)}$ and zero-points $z_x^{(l)}, z_W^{(l)}$
Ensure: Final float output $\hat{y}^{(L)}$

$x^{(0)} \leftarrow x$
for $l = 1$ to L **do**
  $x_{\text{int}}^{(l-1)} \leftarrow \text{round}\, x^{(l-1)}/s_x^{(l)} + z_x^{(l)}$
  $W_{\text{int}}^{(l)} \leftarrow \text{round}\, W^{(l)}/s_W^{(l)} + z_W^{(l)}$
  $y_{\text{int}}^{(l)} \leftarrow \sum_i W_{\text{int},i}^{(l)} \cdot x_{\text{int},i}^{(l-1)}$
  $\hat{y}^{(l)} \leftarrow s_W^{(l)} s_x^{(l)} \cdot y_{\text{int}}^{(l)} + b^{(l)}$
  $x^{(l)} \leftarrow \hat{y}^{(l)}$
end for
return $\hat{y}^{(L)}$

Algorithm 1 describes the forward inference process of a quantized neural network. It begins by quantizing the real-valued inputs and weights of each layer using the corresponding scale and zero-point parameters. Next, it performs efficient integer dot products (multiply-accumulate operations) in the quantized domain. The resulting integer accumulation is then dequantized back to floating-point by applying the scales and adding the bias. This dequantized output serves as the input to the next layer. After all L layers are processed, the final floating-point output $\hat{y}^{(L)}$ is returned. This procedure ensures that the bulk of computations are performed in the integer domain, yielding significant savings in memory and computation while preserving model accuracy.

4 Simulation and Evaluation Settings

4.1 Dataset

To evaluate the quantization methods in a controlled setting, we manually selected a balanced subset of 100 images from the Animals10 dataset [4]. This dataset includes 10 different animal categories: Brittany Spaniel, Horse, Ox, Tiger cat, Barn spider, Butterfly, Elephant, Rooster, Sheep, and Squirrel . From each class, we selected 10 representative images, ensuring an equal distribution across categ ories. The images were resized to 224x224 pixels and normalized using the ImageNet mean and standard deviation.



Figure 3: Example images from the Animals10 dataset [4]: Brittany Spaniel (left) and Horse (right).

4.2 Quantization Configurations

Table 3 summarizes the quantization configurations evaluated in our experiments. Each configuration is defined by its bit-width (8 or 16), symmetry type (symmetric or asymmetric), and clamping strategy. These settings were chosen to reflect common and advanced quantization approaches as described in relevant literature.

Table 3: Quantization configurations tested in our experiments

Bit Width	Symmetry Type	Clamping Method
8	Symmetric	None
8	Symmetric	Percentile (1–99%)
8	Symmetric	Percentile (0.1–99.9%)
8	Symmetric	Std Dev ($\pm 3\sigma$)
8	Symmetric	Std Dev ($\pm 4\sigma$)
16	Symmetric	None
16	Symmetric	Percentile (1–99%)
16	Symmetric	Percentile (0.1–99.9%)
16	Symmetric	Std Dev ($\pm 3\sigma$)
16	Symmetric	Std Dev ($\pm 4\sigma$)
8	Asymmetric	None
8	Asymmetric	Percentile (1–99%)
8	Asymmetric	Percentile (0.1–99.9%)
8	Asymmetric	Std Dev ($\pm 3\sigma$)
8	Asymmetric	Std Dev ($\pm 4\sigma$)
16	Asymmetric	None
16	Asymmetric	Percentile (1–99%)
16	Asymmetric	Percentile (0.1–99.9%)
16	Asymmetric	Std Dev ($\pm 3\sigma$)
16	Asymmetric	Std Dev ($\pm 4\sigma$)

4.3 Evaluation Metrics

Mean Absolute Error *(MAE)*- The *mean error* measures the average of the absolute differences between predicted and true values:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error *(MSE)*- The *mean squared error* calculates the average of the squared differences between predicted and true values. It penalizes larger errors more heavily and is commonly used in regression tasks:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Accuracy- Measures the proportion of correct predictions out of all predictions:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

Here, TP, TN, FP, and FN correspond to the elements of the *confusion matrix*, as summarized below:

Table 4: Confusion matrix terminology for binary classification.

	Predicted Positive	**Predicted Negative**
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

4.4 Experimental Protocol

In this work, we evaluate integer-based quantization configurations to systematically reduce the size of network weights through integer-based quantization, thereby significantly lowering the computational and memory requirements of deep neural networks. By representing the weights in a compact format, models can be executed efficiently even on resource-constrained devices such as edge hardware, mobile systems, and embedded platforms. This reduction in model size not only accelerates inference but also decreases power consumption, which is critical for real-time applications.

To identify the quantization scheme that achieves the best balance between efficiency and accuracy, we evaluate multiple controlled configurations. Specifically, our study examines the effect of three key factors:

Symmetry type - symmetric versus asymmetric quantization,

Bit-width - INT8 and INT16 representations,

Clamping strategy - no clamping, percentile-based thresholds, and standard deviation multipliers.

By systematically combining these settings, we aim to highlight which configuration minimizes error while preserving accuracy. This structured approach ensures that the proposed method is not only theoretically motivated but also empirically validated through reproducible experiments.

4.5 Implementation Details

We implemented our quantization framework using PyTorch. The experiments used standard image classification models, primarily ResNet-18. For each configuration (bit-width, clamping method, and symmetry type), we performed inference and collected evaluation metrics, including classification accuracy, mean squared error (MSE), and mean absolute error (MAE). The full implementation is available on GitHub. All our code and scripts are publicly available at: `https://github.com/NoamCohen12/final-project`.

Experimental Setup. All configurations were evaluated under identical conditions: the same dataset and model (ResNet-18) were used across all runs. The FP32 model serves as the baseline, against which quantized versions are compared. The only factors that vary between runs are the quantization settings, namely the choice of symmetric vs.

asymmetric mapping and the use of clamping (percentile- or standard-deviation-based) versus no clamping. This ensures that any observed differences in accuracy or error metrics are attributable solely to the quantization configuration.

5 Results and Evaluation

We evaluated accuracy (top-1 classification), mean squared error (MSE), and mean absolute error (MAE) for different quantization methods. These metrics are formally defined below:

5.1 Quantization Results

Results indicate that quantization **without clamping** yields the most accurate outcomes, with minimal error across both symmetric and asymmetric mappings. Introducing clamping strategies (percentile or standard deviation based) substantially increases the error, highlighting the trade-off between memory efficiency and numerical precision.

5.2 MAE Results

INT8

Table 5: INT8 MAE under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**9.61e-05**	7.52e-04	1.95e-03	1.94e-03	1.55e-03
asymmetric	**7.47e-05**	7.55e-04	1.95e-03	1.94e-03	1.55e-03

INT16

Table 6: INT16 MAE under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**6.24e-06**	7.56e-04	1.95e-03	1.94e-03	1.55e-03
asymmetric	**6.29e-06**	7.55e-04	1.95e-03	1.94e-03	1.55e-03

5.3 MSE Results

INT8

Table 7: INT8 MSE under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**3.84e-06**	1.70e-04	5.64e-04	5.50e-04	4.46e-04
asymmetric	**2.54e-06**	1.71e-04	5.65e-04	5.50e-04	4.46e-04

INT16

Table 8: INT16 MSE under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**2.30e-08**	1.71e-04	5.64e-04	5.50e-04	4.45e-04
asymmetric	**2.34e-08**	1.71e-04	5.64e-04	5.50e-04	4.45e-04

5.4 Accuracy Results

INT8

Table 9: INT8 accuracy under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**0.99**	0.61	0.0	0.0	0.19
asymmetric	**0.95**	0.63	0.0	0.0	0.21

INT16

Table 10: INT16 accuracy under different clamping strategies

Sym / asym	0 clamp	0.1% clamp	1% clamp	Std 3σ	Std 4σ
symmetric	**1.0**	0.63	0.0	0.0	0.20
asymmetric	**1.0**	0.63	0.0	0.0	0.20

5.5 Visualization

To complement the tabular results, Figures 6 and 5 show grouped bar charts comparing symmetric and asymmetric quantization across clamping strategies.

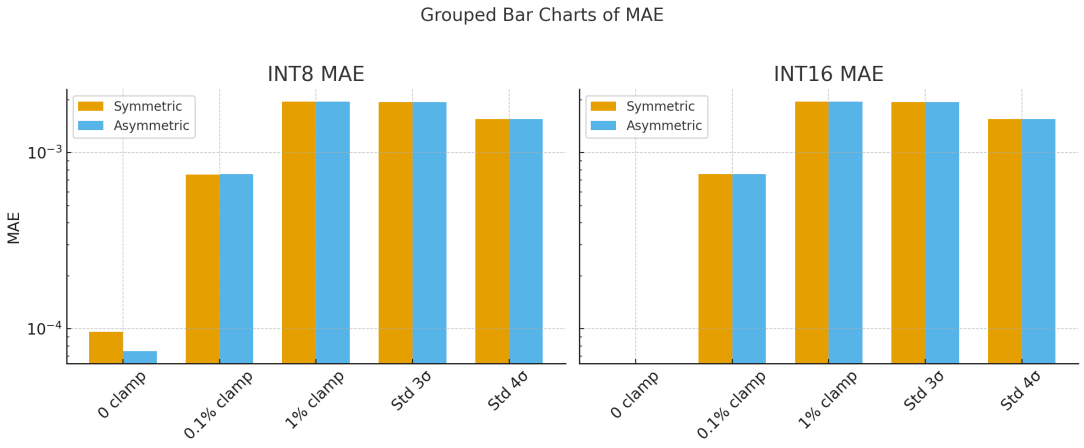


Figure 4: MAE for INT8 and INT16 under different clamping strategies.

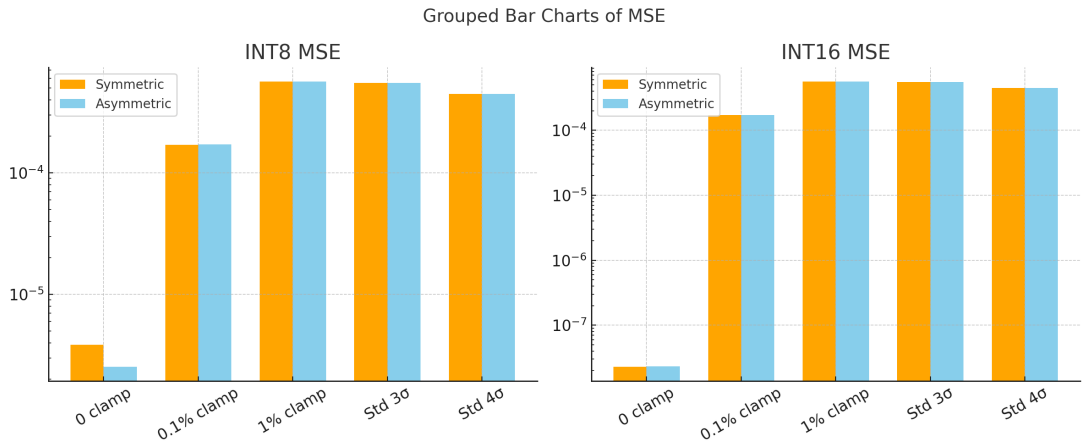


Figure 5: MSE for INT8 and INT16 under different clamping strategies.

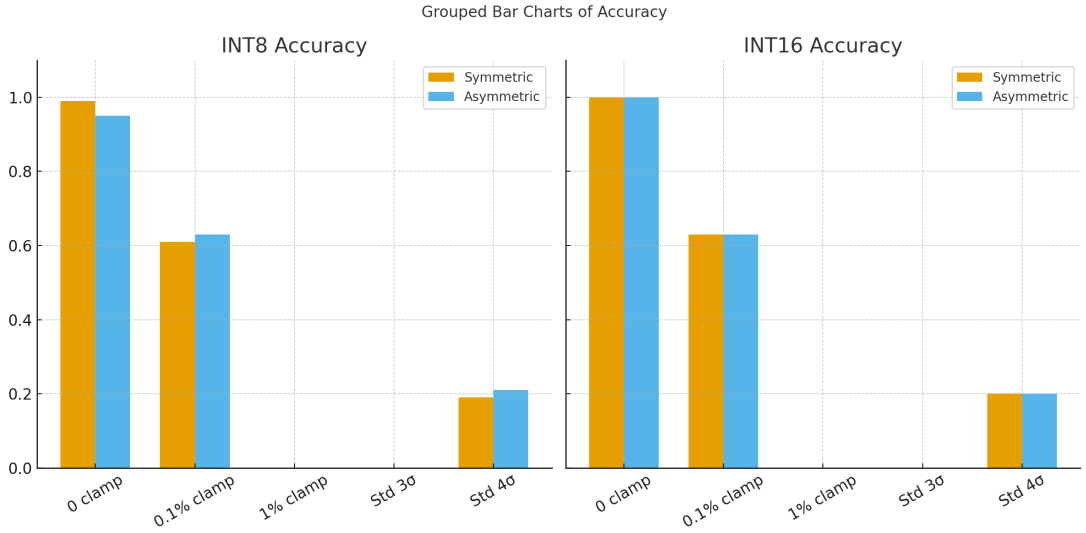


Figure 6: Accuracy for INT8 and INT16 under different clamping strategies.

5.6 Discussion of Results

The observed results can be explained by analyzing the data distribution. Our pixel-level analysis (over 14M pixels) showed that the dataset is well-behaved, with mean intensity $\mu \approx 123.6$, standard deviation $\sigma \approx 63.9$, and extreme values lying only about 2σ from the mean. This indicates the absence of heavy-tailed outliers. Consequently, clamping at percentile or $k\sigma$ thresholds does not remove true anomalies but instead clips valid values, narrowing the effective quantization range and introducing error.

This behavior is evident in the tabular and visualized results. Quantization **without clamping** consistently yielded the lowest MAE and MSE and the highest accuracy, across both symmetric and asymmetric mappings. In contrast, introducing clamping strategies caused a sharp degradation in accuracy, with INT8 accuracy dropping to nearly 0.0 under 1% or 3σ clamping.

The difference between INT8 and INT16 further highlights sensitivity to range misspecification. With only 256 discrete levels, **INT8** is highly sensitive to any reduction of the quantization interval, making it vulnerable to unnecessary clamping. Meanwhile, **INT16**, offering 65,536 levels, maintains much lower errors in the no-clamp setting (10^{-6} to 10^{-8}) and is more robust overall, though still affected by clamping.

Figure 7 illustrates the pixel intensity distribution, together with μ and the $\pm 2\sigma, \pm 3\sigma$ bounds. The lack of extreme outliers confirms that clamping is unnecessary for this dataset, and directly explains the performance degradation observed in Figures. 4–6.

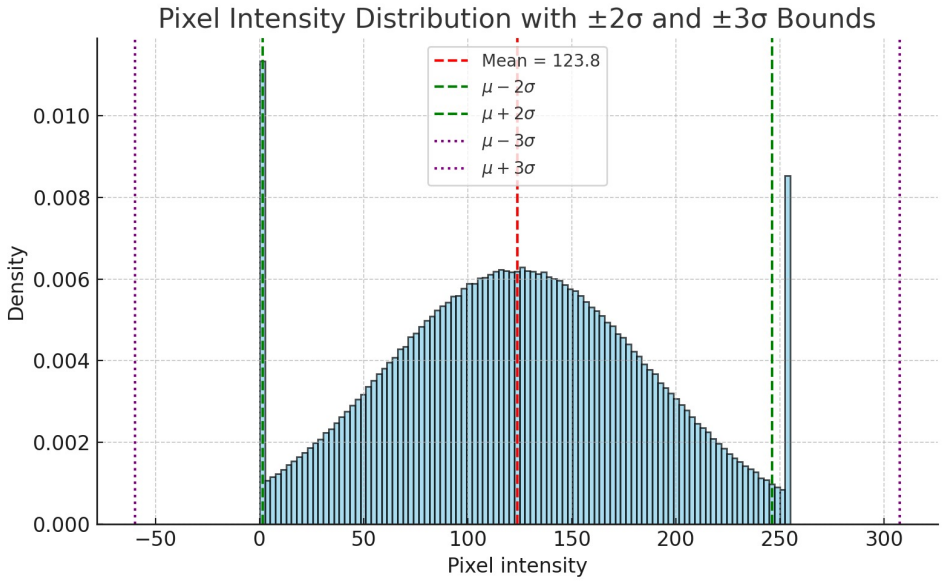


Figure 7: Pixel intensity distribution across the dataset, with mean and $\pm 2\sigma$, $\pm 3\sigma$ bounds. All values lie within 2σ, confirming the absence of heavy-tailed outliers and explaining why clamping degrades performance.

6 Conclusions and Future Work

Conclusion. In this study, we examined different quantization schemes, comparing symmetric versus asymmetric approaches and evaluating the impact of clipping. Our findings demonstrate that *symmetric quantization without clipping* consistently achieves the highest accuracy while minimizing errors across the tested scenarios. Moreover, this method offers a favorable balance between accuracy and efficiency, leading to faster inference times and reduced computational demands. These properties make it particularly suitable for deploying deep neural networks on edge devices with limited hardware resources.

In this work, we systematically evaluated integer-based quantization strategies (INT8 and INT16), exploring symmetric and asymmetric schemes combined with various clamping methods. Using ResNet-18 on the Animals10 dataset, we demonstrated the practical trade-offs between efficiency and accuracy, bridging the gap between theoretical advances and real-world deployment.

Future Work. Although our results indicate that symmetric quantization without clipping is generally the most effective choice, it is evident that this approach may not be optimal in all cases. Certain edge cases and model architectures may benefit more from alternative quantization methods. A promising future direction is the development of an adaptive framework capable of learning from diverse examples to determine the optimal quantization strategy for a given model and dataset. Such a system could dynamically select the most suitable quantization method during deployment, rather than relying on a one-size-fits-all solution. This adaptive approach has the potential to maximize both accuracy and resource efficiency, while ensuring robust performance across a wide range of real-world scenarios.

References

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)," *IEEE Computer*, vol. 44, no. 7, pp. 32–34, 2011.

[2] I. Cohen and G. Einziger, "Floating-floating point: a highly accurate number representation with flexible counting ranges," *IEEE Transactions on Computers*, 2025.

[3] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, "A White Paper on Neural Network Quantization," arXiv:2106.08295 [cs.LG], Jun. 2021.

[4] A. Corrado, *Animals-10 Image Dataset*, Kaggle, 2019. Available: `https://www.kaggle.com/datasets/alessiocorrado99/animals10`

[5] Y. Xu, L. Xie, X. Gu, X. Chen, H. Chang, H. Zhang, Z. Chen, X. Zhang, and Q. Tian, "QA-LoRA: Quantization-Aware Low-Rank Adaptation of Large Language Models," in *Proc. ICLR*, 2024.

[6] T. Chen, Z. Zhang, A. Jaiswal, S. Liu, and Z. Wang, "Sparse MoE as the New Dropout: Scaling Dense and Self-Slimmable Transformers," University of Texas at Austin, 2024.

[7] W. Kahan, "IEEE standard 754 for binary floating-point arithmetic," *Lecture Notes on the Status of IEEE*, vol. 754, no. 94720-1776, p. 11, 1996.

[8] NVIDIA, "TensorFloat-32 in the A100 GPU Accelerates AI Training, HPC up to 20x," NVIDIA Blog, 2020. Available: `https://blogs.nvidia.com/blog/tensorfloat-32-precision-format/`

[9] K. Curcio, et al., "Introducing LCOAI: A Standardized Economic Metric for Evaluating AI Deployment Costs," *arXiv preprint arXiv:2501.00001*, 2025. Available: `https://arxiv.org/abs/2509.02596`